Visualizador 3D de Trajetórias de Robôs Subaquáticos

Fundamentos matemáticos, referenciais e vista em 1.ª pessoa

Documentação das bibliotecas matemáticas (C++ OOP) e da aplicação (Python)

1 de outubro de 2026

Índice

1 Introdução e convenções **3**

1.1 Arquitetura em duas linguagens . . . 3

1.2 Regras de notação . . . 3

2 Referenciais **3**

3 Geodesia: de latitude/longitude para coordenadas locais **4**

3.1 Latitude, longitude, altura → ECEF . . . 4

3.2 ECEF → NED . . . 5

3.3 NED → azimute, elevação, distância (AER) . . . 6

4 Rotações **6**

4.1 Rotação em 2D . . . 6

4.2 Rotações elementares em 3D . . . 6

4.3 Propriedades de uma matriz de rotação . . . 7

4.4 Orientação do drone: guinada, arfagem, rolamento . . . 7

5 Mudança de referencial e vista local **8**

6 Cinemática: como a trajetória evolui no tempo **8**

6.1 Posição a partir da velocidade . . . 8

6.2 Orientação a partir da velocidade angular . . . 9

6.3 Orientação deduzida da trajetória . . . 10

6.4 Interpolação no Mission Replay . . . 10

7 Câmara em 1.ª pessoa **10**

7.1 Referencial da câmara e ligação ao corpo . . . 10

7.2 Do mundo para a câmara . . . 11

7.3 Projeção em perspetiva (modelo pinhole) . . . 11

7.4 Distância focal a partir do campo de visão 12
7.5 Quando um ponto é visível 12
7.6 Ligação a um motor gráfico 3D 12

8 Exemplo numérico completo **12**

9 Arquitetura do software **13**
9.1 Classes C++ e a matemática que implementam 13
9.2 Esboço da interface 13
9.3 Ligação Python ↔ C++ 14
9.4 Testes que validam a matemática 14

10 Resumo das fórmulas **15**

1 Introdução e convenções

Este documento explica a matemática usada pelo visualizador 3D de trajetórias e pela vista em 1.ª pessoa (o ponto de vista de cada drone). Cada fórmula é *deduzida passo a passo* a partir de definições simples, sem saltar contas.

1.1 Arquitetura em duas linguagens

- **Python**: aplicação, leitura de configuração, interface e visualização.
- **C++ (orientado a objetos)**: toda a matemática (vetores, matrizes, rotações, referenciais, geodesia, câmara). É uma biblioteca reutilizável, chamada a partir do Python.

1.2 Regras de notação

Para que tudo seja fácil de acompanhar, usa-se sempre a mesma notação:

- Posições e vetores são **vetores coluna** de 3 componentes, escritos com x, y, z: $\mathbf{p} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$.
- Todas as matrizes são de dimensão 3×3 (**não** se usam matrizes homogéneas 4×4). Uma mudança de referencial escreve-se com uma matriz 3×3 *mais* um vetor de translação, de forma explícita.
- Ângulos em radianos. cos e sin abreviam-se por c e s nas matrizes grandes, com o ângulo no índice: $c_\psi = \cos\psi$.
- O índice inferior indica o referencial em que um vetor está escrito: $\mathbf{p}_w$ (mundo), $\mathbf{p}_b$ (corpo do drone), $\mathbf{p}_c$ (câmara).
- $\mathbf{R}_{wb}$ é a matriz que converte *do referencial b para o referencial w*. Lê-se da direita para a esquerda: $\mathbf{p}_w = \mathbf{R}_{wb}\,\mathbf{p}_b + \ldots$

Símbolo	Significado
$\mathbf{p}_w = (x, y, z)^\mathsf{T}$	posição no referencial do mundo (NED)
$\mathbf{t}_{wb}$	posição da origem do corpo b, escrita em w
$\mathbf{R}_{wb}$	matriz 3×3 de rotação de b para w
ϕ, θ, ψ	roll (rolamento), pitch (arfagem), yaw (guinada)
$\mathbf{v}$, ω	velocidade linear, velocidade angular
f_x, f_y, c_x, c_y	parâmetros da câmara (distância focal e ponto principal)

Tabela 1: Símbolos usados no documento.

2 Referenciais

Um *referencial* é uma origem mais três eixos perpendiculares entre si. Usam-se quatro:

1. **ECEF** (Earth-Centered, Earth-Fixed): origem no centro da Terra, z pelo eixo de rotação (Polo Norte), x no cruzamento do equador com o meridiano de Greenwich, y completa o sistema direito. Serve para coordenadas globais (GPS).
2. **NED** (North–East–Down): referencial local do mundo, com origem num ponto de referência. x aponta para Norte, y para Este, z para baixo. É o usado na simulação. Como z cresce para baixo, **a profundidade de um robô subaquático é diretamente a sua coordenada z.**

3. **Corpo** (b): preso ao drone. x_b para a frente, y_b para a direita, z_b para baixo.

4. **Câmara** (c): preso à câmara do drone. x_c para a direita da imagem, y_c para baixo da imagem, z_c para a frente (eixo ótico).

Todos são sistemas **direitos**: $\hat{x} \times \hat{y} = \hat{z}$.

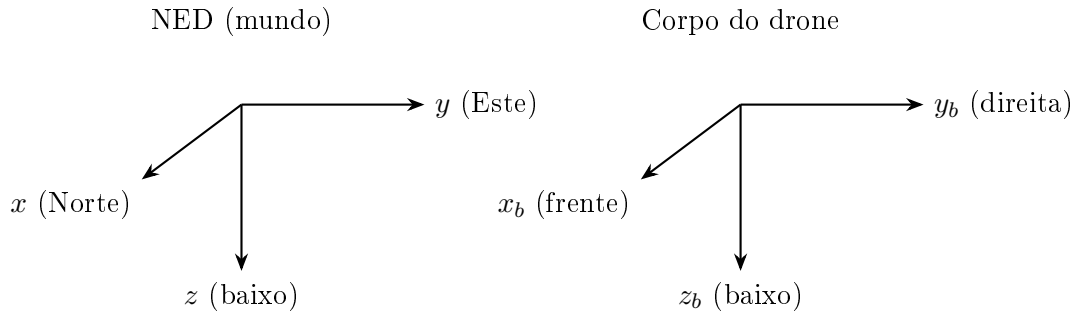


Figura 1: Os eixos do mundo (NED) e do corpo têm a mesma disposição; só diferem na origem e na orientação (que o drone pode rodar).

3 Geodesia: de latitude/longitude para coordenadas locais

Quando os dados chegam como latitude φ, longitude λ e altura h (GPS), é preciso convertê-los para o referencial NED.

3.1 Latitude, longitude, altura → ECEF

A Terra é modelada por um elipsoide (WGS84) com semieixo maior a (equador) e semieixo menor b (polos). A excentricidade e define-se por

$$e^2 = 1 - \frac{b^2}{a^2} \quad \Longrightarrow \quad b^2 = a^2(1 - e^2). \tag{1}$$

Dedução 3.1 (Coordenadas ECEF de um ponto na superfície)**. Passo 1 — Elipse do meridiano.** Cortando o elipsoide por um plano que contém o eixo z, obtém-se uma elipse. Chamamos ρ à distância ao eixo z (no plano do equador) e z à altura. A elipse é

$$\frac{\rho^2}{a^2} + \frac{z^2}{b^2} = 1.$$

Passo 2 — Declive da normal. A latitude geodésica φ é o ângulo entre a normal à superfície e o plano do equador. Derivando implicitamente a elipse:

$$\frac{2\rho}{a^2} + \frac{2z}{b^2}\frac{dz}{d\rho} = 0 \implies \frac{dz}{d\rho} = -\frac{b^2\rho}{a^2 z}.$$

Este é o declive da tangente. A normal é perpendicular, logo o seu declive é o simétrico do inverso:

$$\tan\varphi = \frac{a^2 z}{b^2 \rho} \implies z = \frac{b^2}{a^2}\,\rho\tan\varphi = (1 - e^2)\,\rho\tan\varphi,$$

onde se usou (1).

Passo 3 — Substituir na elipse. Metendo z na equação da elipse e usando $b^2 = a^2(1-e^2)$:

$$\frac{\rho^2}{a^2} + \frac{(1-e^2)^2\rho^2\tan^2\varphi}{a^2(1-e^2)} = 1 \implies \frac{\rho^2}{a^2}\Big[1 + (1-e^2)\tan^2\varphi\Big] = 1.$$

Passo 4 — Resolver em ρ. Multiplicando o parêntesis por $\cos^2\varphi$ em cima e em baixo:

$$\rho^2 = \frac{a^2\cos^2\varphi}{\cos^2\varphi + (1-e^2)\sin^2\varphi} = \frac{a^2\cos^2\varphi}{1 - e^2\sin^2\varphi},$$

pois $\cos^2\varphi + \sin^2\varphi = 1$. Definindo

$$N = \frac{a}{\sqrt{1 - e^2\sin^2\varphi}}, \tag{2}$$

fica $\rho = N\cos\varphi$ e, do passo 2, $z = (1-e^2)\,N\cos\varphi\tan\varphi = N(1-e^2)\sin\varphi$.

Passo 5 — Rodar para a longitude. O ponto está no plano do meridiano; rodar em torno de z pelo ângulo λ dá $x = \rho\cos\lambda$, $y = \rho\sin\lambda$.

Passo 6 — Acrescentar a altura h. Subir h ao longo da normal, cuja direção unitária é $(\cos\varphi\cos\lambda,\ \cos\varphi\sin\lambda,\ \sin\varphi)^{\mathsf{T}}$ (ver Dedução 3.2).

$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix}_{\text{ECEF}} = \begin{pmatrix} (N+h)\cos\varphi\cos\lambda \\ (N+h)\cos\varphi\sin\lambda \\ \big(N(1-e^2)+h\big)\sin\varphi \end{pmatrix}, \quad N = \frac{a}{\sqrt{1 - e^2\sin^2\varphi}}$$

Para um robô subaquático, h é negativo (abaixo do nível do mar): $h = -\text{profundidade}$.

3.2 ECEF → NED

Dedução 3.2 (Eixos locais Norte, Este, Baixo em ECEF)**.** Seja $\mathbf{r}(\varphi,\lambda)$ a posição ECEF de um ponto à superfície (com $h = 0$). As direções locais obtêm-se derivando $\mathbf{r}$ e normalizando.

Este é a direção em que só λ aumenta:

$$\frac{\partial\mathbf{r}}{\partial\lambda} = \begin{pmatrix} -\rho\sin\lambda \\ \rho\cos\lambda \\ 0 \end{pmatrix} \implies \hat{e} = \begin{pmatrix} -\sin\lambda \\ \cos\lambda \\ 0 \end{pmatrix}$$

(divide-se pelo comprimento ρ).

Norte é a direção em que só φ aumenta. Derivando $\rho = N\cos\varphi$ e $z = N(1-e^2)\sin\varphi$ em ordem a φ e normalizando (o resultado é o vetor tangente ao meridiano):

$$\hat{n} = \begin{pmatrix} -\sin\varphi\cos\lambda \\ -\sin\varphi\sin\lambda \\ \cos\varphi \end{pmatrix}.$$

Verificação: $\hat{n}\cdot\hat{e} = \sin\varphi\sin\lambda\cos\lambda - \sin\varphi\sin\lambda\cos\lambda + 0 = 0$ (perpendiculares) e $|\hat{n}|^2 = \sin^2\varphi(\cos^2\lambda + \sin^2\lambda) + \cos^2\varphi = 1$.

Cima (normal à superfície) é $\hat{u} = \hat{e}\times\hat{n}$. Calculando o produto vetorial:

$$\hat{u} = \begin{pmatrix} \cos\varphi\cos\lambda \\ \cos\varphi\sin\lambda \\ \sin\varphi \end{pmatrix}, \qquad \hat{d} = -\hat{u}.$$

Dedução 3.3 (Matriz ECEF → NED)**.** Seja $\mathbf{q} = \mathbf{p}_{\text{ECEF}} - \mathbf{p}_{\text{ref}}$ o vetor do ponto de referência até ao ponto. A componente de $\mathbf{q}$ ao longo de cada eixo local obtém-se com o produto escalar:

$$x_N = \hat{n} \cdot \mathbf{q}, \qquad y_E = \hat{e} \cdot \mathbf{q}, \qquad z_D = \hat{d} \cdot \mathbf{q}.$$

Empilhar as três linhas dá uma matriz 3×3.

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix}_{\text{NED}} = \underbrace{\begin{pmatrix} -\sin\varphi_0 \cos\lambda_0 & -\sin\varphi_0 \sin\lambda_0 & \cos\varphi_0 \\ -\sin\lambda_0 & \cos\lambda_0 & 0 \\ -\cos\varphi_0 \cos\lambda_0 & -\cos\varphi_0 \sin\lambda_0 & -\sin\varphi_0 \end{pmatrix}}_{\mathbf{R}_{\text{NED,ECEF}}} (\mathbf{p}_{\text{ECEF}} - \mathbf{p}_{\text{ref}})$$

φ_0, λ_0 são a latitude e a longitude do ponto de referência.

3.3 NED → azimute, elevação, distância (AER)

Com (x, y, z) em NED, define-se a distância horizontal $\rho_h = \sqrt{x^2 + y^2}$. Então:

$$\text{distância} = \sqrt{x^2 + y^2 + z^2}, \qquad \text{azimute} = \text{atan2}(y,\, x), \qquad \text{elevação} = \text{atan2}(-z,\, \rho_h).$$

O azimute é medido a partir do Norte, para Este. O sinal de menos em z existe porque z aponta para baixo e a elevação é positiva para cima. O uso de atan2 (em vez de $\arctan(y/x)$) garante o quadrante correto.

4 Rotações

4.1 Rotação em 2D

Dedução 4.1 (Matriz de rotação 2D)**.** Um ponto à distância r da origem, com ângulo α em relação ao eixo x, tem coordenadas

$$x = r\cos\alpha, \qquad y = r\sin\alpha.$$

Rodar o ponto de θ (sentido anti-horário) soma θ ao ângulo: $x' = r\cos(\alpha+\theta)$, $y' = r\sin(\alpha+\theta)$. Pelas fórmulas da soma de ângulos:

$$\begin{aligned} x' &= r\cos\alpha\cos\theta - r\sin\alpha\sin\theta = x\cos\theta - y\sin\theta, \\ y' &= r\sin\alpha\cos\theta + r\cos\alpha\sin\theta = x\sin\theta + y\cos\theta. \end{aligned}$$

Em forma matricial:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}.$$

4.2 Rotações elementares em 3D

Uma rotação em torno de um eixo deixa a coordenada desse eixo inalterada e roda as outras duas com a matriz 2D. Rodar em torno de z por ψ (guinada):

$$\mathbf{R}_z(\psi) = \begin{pmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Em torno de x por ϕ (a coordenada x fica; rodam-se y e z, por esta ordem, sentido direito):

$$\mathbf{R}_x(\phi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi & c_\phi \end{pmatrix}.$$

Em torno de y por θ: pela regra da mão direita, o par que roda é (z, x) (permutação cíclica $x \to y \to z \to x$). Por isso o sinal do seno troca de lado em relação às outras:

$$\mathbf{R}_y(\theta) = \begin{pmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{pmatrix}.$$

4.3 Propriedades de uma matriz de rotação

Dedução 4.2 (A inversa é a transposta)**.** As colunas de $\mathbf{R}$ são os eixos do novo referencial escritos no antigo: $\mathbf{R} = [\hat{a}_1\ \hat{a}_2\ \hat{a}_3]$. Os eixos são unitários e perpendiculares: $\hat{a}_i \cdot \hat{a}_j = 1$ se $i = j$ e 0 caso contrário. O elemento (i, j) de $\mathbf{R}^\mathsf{T}\mathbf{R}$ é a linha i de $\mathbf{R}^\mathsf{T}$ (que é a coluna i de $\mathbf{R}$) vezes a coluna j de $\mathbf{R}$, isto é, $\hat{a}_i \cdot \hat{a}_j$. Logo

$$\mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{I} \quad \Longrightarrow \quad \mathbf{R}^{-1} = \mathbf{R}^\mathsf{T}.$$

Além disso $\det \mathbf{R} = +1$ (sistema direito). Consequência prática: **inverter uma rotação é só transpor a matriz**, sem inversão numérica.

4.4 Orientação do drone: guinada, arfagem, rolamento

A orientação do drone descreve-se por três ângulos, aplicados nesta ordem (convenção ZYX, a usual em veículos): primeiro roda-se ϕ em torno de x (roll), depois θ em torno de y (pitch), por fim ψ em torno de z (yaw).

$$\mathbf{R}_{wb} = \mathbf{R}_z(\psi)\, \mathbf{R}_y(\theta)\, \mathbf{R}_x(\phi). \tag{3}$$

Dedução 4.3 (Matriz completa $\mathbf{R}_{wb}$)**. Passo 1:** calcular $\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$, linha a linha.

$$\mathbf{R}_y\mathbf{R}_x = \begin{pmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi & c_\phi \end{pmatrix} = \begin{pmatrix} c_\theta & s_\theta s_\phi & s_\theta c_\phi \\ 0 & c_\phi & -s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{pmatrix}.$$

(por exemplo, elemento $(1,2)$: $c_\theta \cdot 0 + 0 \cdot c_\phi + s_\theta \cdot s_\phi$.)

Passo 2: multiplicar por $\mathbf{R}_z(\psi)$ à esquerda. A linha 3 de $\mathbf{R}_z$ é $(0, 0, 1)$, logo a linha 3 do resultado não muda. A linha 1 do resultado é c_ψ [linha 1]$-s_\psi$ [linha 2] e a linha 2 é s_ψ [linha 1]$+c_\psi$ [linha 2]:

$$\boxed{\mathbf{R}_{wb} = \begin{pmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{pmatrix}}$$

Significado das colunas. A coluna 1 é o eixo x_b (a frente do drone) escrito em NED: $\hat{x}_b = (c_\psi c_\theta,\ s_\psi c_\theta,\ -s_\theta)^\mathsf{T}$. A coluna 2 é $\hat{y}_b$ e a coluna 3 é $\hat{z}_b$.

Dedução 4.4 (Extrair os ângulos de uma matriz)**.** Comparando com a matriz acima:

$$\begin{aligned} R_{31} = -s_\theta \qquad & \Rightarrow\ \theta = \operatorname{atan2}\big(-R_{31}, \sqrt{R_{11}^2 + R_{21}^2}\big), \\ R_{21}/R_{11} = s_\psi/c_\psi = \tan\psi \qquad & \Rightarrow\ \psi = \operatorname{atan2}(R_{21}, R_{11}), \\ R_{32}/R_{33} = s_\phi/c_\phi = \tan\phi \qquad & \Rightarrow\ \phi = \operatorname{atan2}(R_{32}, R_{33}). \end{aligned}$$

Se $\theta = \pm\pi/2$ então $c_\theta = 0$ e ψ, ϕ deixam de ser separáveis (*gimbal lock*). Os drones subaquáticos raramente chegam a $\pm 90°$ de arfagem, mas a biblioteca trata este caso à parte.

5 Mudança de referencial e vista local

Dedução 5.1 (De corpo para mundo)**.** Seja $\mathbf{t}_{wb}$ a posição da origem do corpo, escrita no mundo, e $\mathbf{R}_{wb}$ a orientação. Um ponto $\mathbf{p}_b$ (escrito no corpo) é: (i) rodado para ficar com eixos alinhados com o mundo, $\mathbf{R}_{wb}\mathbf{p}_b$; (ii) deslocado pela posição da origem. Então

$$\mathbf{p}_w = \mathbf{R}_{wb}\,\mathbf{p}_b + \mathbf{t}_{wb}. \tag{4}$$

Dedução 5.2 (De mundo para corpo — a vista local)**.** Isolar $\mathbf{p}_b$ na equação anterior. Primeiro subtrai-se $\mathbf{t}_{wb}$:

$$\mathbf{p}_w - \mathbf{t}_{wb} = \mathbf{R}_{wb}\,\mathbf{p}_b.$$

Multiplica-se à esquerda por $\mathbf{R}_{wb}^{-1} = \mathbf{R}_{wb}^{\mathsf{T}}$ (propriedade provada na secção anterior):

$$\boxed{\mathbf{p}_b = \mathbf{R}_{wb}^{\mathsf{T}}\,(\mathbf{p}_w - \mathbf{t}_{wb})}$$

É exatamente isto que a **Local View** faz para cada ponto da cena: subtrai a posição do robô selecionado e roda com a transposta da sua orientação. Passa-se a ver o mundo como o robô o vê.

Dedução 5.3 (Composição de referenciais)**.** Se o corpo b está no mundo w, e a câmara c está no corpo b, então $\mathbf{p}_b = \mathbf{R}_{bc}\mathbf{p}_c + \mathbf{t}_{bc}$ e $\mathbf{p}_w = \mathbf{R}_{wb}\mathbf{p}_b + \mathbf{t}_{wb}$. Substituindo a primeira na segunda:

$$\mathbf{p}_w = \mathbf{R}_{wb}\big(\mathbf{R}_{bc}\mathbf{p}_c + \mathbf{t}_{bc}\big) + \mathbf{t}_{wb} = \underbrace{\mathbf{R}_{wb}\mathbf{R}_{bc}}_{\mathbf{R}_{wc}}\,\mathbf{p}_c + \underbrace{\mathbf{R}_{wb}\mathbf{t}_{bc} + \mathbf{t}_{wb}}_{\mathbf{t}_{wc}}.$$

Ou seja, **rotações multiplicam-se** e a translação da câmara no mundo é a do corpo mais a translação da câmara *rodada* pela orientação do corpo.

6 Cinemática: como a trajetória evolui no tempo

6.1 Posição a partir da velocidade

A velocidade é a derivada da posição. Em componentes:

$$\frac{dx}{dt} = v_x, \qquad \frac{dy}{dt} = v_y, \qquad \frac{dz}{dt} = v_z.$$

Se a velocidade é medida no corpo ($\mathbf{v}_b$, por exemplo velocidade do DVL), converte-se para o mundo com a Dedução 5.1 (só a parte da rotação, pois uma velocidade é um vetor livre): $\mathbf{v}_w = \mathbf{R}_{wb}\mathbf{v}_b$.

Dedução 6.1 (Atualização discreta (Euler))**.** Desenvolvendo $\mathbf{p}(t + \Delta t)$ em série de Taylor em torno de t:

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t)\,\Delta t + \tfrac{1}{2}\mathbf{a}(t)\,\Delta t^2 + \cdots$$

Truncando no primeiro termo (erro de ordem Δt^2):

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k\,\Delta t.$$

Se a velocidade for constante no intervalo, é exato.

6.2 Orientação a partir da velocidade angular

Dedução 6.2 (Equação diferencial da rotação)**. Passo 1.** Para qualquer instante, $\mathbf{R}\mathbf{R}^{\mathsf{T}} = \mathbf{I}$. Derivando em ordem ao tempo (regra do produto):

$$\dot{\mathbf{R}}\,\mathbf{R}^{\mathsf{T}} + \mathbf{R}\,\dot{\mathbf{R}}^{\mathsf{T}} = \mathbf{0}.$$

Passo 2. Seja $\mathbf{S} = \dot{\mathbf{R}}\mathbf{R}^{\mathsf{T}}$. Como $(\dot{\mathbf{R}}\mathbf{R}^{\mathsf{T}})^{\mathsf{T}} = \mathbf{R}\dot{\mathbf{R}}^{\mathsf{T}}$, a equação do passo 1 diz $\mathbf{S} + \mathbf{S}^{\mathsf{T}} = \mathbf{0}$, ou seja, $\mathbf{S}$ é *antissimétrica*. Uma matriz 3×3 antissimétrica tem a forma

$$[\omega]_\times = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix}.$$

Só tem 3 números independentes, que são as componentes do vetor ω (velocidade angular).

Passo 3. Verificar que esta matriz faz o produto vetorial:

$$[\omega]_\times \mathbf{v} = \begin{pmatrix} -\omega_z v_y + \omega_y v_z \\ \omega_z v_x - \omega_x v_z \\ -\omega_y v_x + \omega_x v_y \end{pmatrix} = \omega \times \mathbf{v}.$$

Passo 4. De $\mathbf{S} = \dot{\mathbf{R}}\mathbf{R}^{\mathsf{T}} = [\omega_w]_\times$, multiplicando à direita por $\mathbf{R}$:

$$\dot{\mathbf{R}} = [\omega_w]_\times\,\mathbf{R}.$$

Se ω for dada no corpo, $\omega_w = \mathbf{R}\omega_b$ e obtém-se a forma equivalente $\dot{\mathbf{R}} = \mathbf{R}\,[\omega_b]_\times$.

Dedução 6.3 (Solução: fórmula de Rodrigues)**.** Com ω_b constante no passo Δt, a EDO $\dot{\mathbf{R}} = \mathbf{R}[\omega_b]_\times$ tem solução

$$\mathbf{R}(t + \Delta t) = \mathbf{R}(t)\,\exp\big([\omega_b]_\times \Delta t\big),$$

onde a exponencial de matriz é definida pela série $\exp(\mathbf{A}) = \mathbf{I} + \mathbf{A} + \frac{1}{2!}\mathbf{A}^2 + \frac{1}{3!}\mathbf{A}^3 + \cdots$

Passo 1. Seja $\alpha = |\omega_b|\Delta t$ (ângulo rodado) e $\mathbf{K} = [\hat{\omega}]_\times$ com $\hat{\omega} = \omega_b/|\omega_b|$ unitário. Então $[\omega_b]_\times \Delta t = \alpha\mathbf{K}$.

Passo 2. Para um eixo unitário, calcula-se diretamente que $\mathbf{K}^3 = -\mathbf{K}$. Logo $\mathbf{K}^4 = -\mathbf{K}^2$, $\mathbf{K}^5 = \mathbf{K}$, e assim por diante: as potências repetem-se com período 4 a menos de sinal.

Passo 3. Agrupar a série por potências de $\mathbf{K}$ e de $\mathbf{K}^2$:

$$\exp(\alpha\mathbf{K}) = \mathbf{I} + \Big(\alpha - \tfrac{\alpha^3}{3!} + \tfrac{\alpha^5}{5!} - \cdots\Big)\mathbf{K} + \Big(\tfrac{\alpha^2}{2!} - \tfrac{\alpha^4}{4!} + \cdots\Big)\mathbf{K}^2.$$

Reconhecem-se as séries de $\sin\alpha$ e de $1 - \cos\alpha$.

$$\boxed{\exp(\alpha\mathbf{K}) = \mathbf{I} + \sin\alpha\,\mathbf{K} + (1 - \cos\alpha)\,\mathbf{K}^2}$$

Exemplo 6.1 (Verificação)**.** $\omega = (0, 0, 1)^{\mathsf{T}}$ rad/s e $\Delta t = \pi/2$ s, logo $\alpha = \pi/2$, $\mathbf{K} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$, $\mathbf{K}^2 = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$. Então $\mathbf{I} + 1\cdot\mathbf{K} + 1\cdot\mathbf{K}^2 = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \mathbf{R}_z(90^\circ)$, como esperado.

Após muitos passos, erros numéricos fazem $\mathbf{R}^{\mathsf{T}}\mathbf{R}$ afastar-se de $\mathbf{I}$. Por isso a biblioteca *re-ortonormaliza* periodicamente a matriz (por exemplo com Gram–Schmidt nas colunas).

6.3 Orientação deduzida da trajetória

Quando só há posições (sem orientação medida), assume-se que o drone aponta para onde se move. O eixo $\hat{x}_b$ deve ser paralelo à velocidade $\mathbf{v} = (v_x, v_y, v_z)^\mathsf{T}$.

Dedução 6.4 (Yaw e pitch a partir da velocidade)**.** Da matriz $\mathbf{R}_{wb}$, a primeira coluna é $\hat{x}_b = (c_\psi c_\theta,\ s_\psi c_\theta,\ -s_\theta)^\mathsf{T}$. Igualando a $\mathbf{v}/|\mathbf{v}|$:

$$c_\psi c_\theta = \frac{v_x}{|\mathbf{v}|}, \quad s_\psi c_\theta = \frac{v_y}{|\mathbf{v}|}, \quad -s_\theta = \frac{v_z}{|\mathbf{v}|}.$$

Dividindo a segunda pela primeira, $\tan\psi = v_y/v_x$. Para o pitch, note-se que $\sqrt{(c_\psi c_\theta)^2 + (s_\psi c_\theta)^2} = |c_\theta|$, logo $c_\theta = \sqrt{v_x^2 + v_y^2}/|\mathbf{v}|$ (para $|\theta| < \pi/2$, $c_\theta \geq 0$). Assim:

$$\boxed{\psi = \operatorname{atan2}(v_y, v_x), \qquad \theta = \operatorname{atan2}\big(-v_z, \sqrt{v_x^2 + v_y^2}\big), \qquad \phi = 0}$$

O roll não é determinável só pela velocidade e fica a 0, a não ser que exista medição. Em NED, descer ($v_z > 0$) dá $\theta < 0$ (nariz para baixo).

6.4 Interpolação no Mission Replay

O slider do replay pede o estado num instante t entre duas amostras $(t_0, \mathbf{p}_0)$ e $(t_1, \mathbf{p}_1)$.

Dedução 6.5 (Interpolação linear)**.** Define-se a fração $s = \dfrac{t - t_0}{t_1 - t_0} \in [0, 1]$. Admitindo velocidade constante entre amostras, a velocidade é $\mathbf{v} = \dfrac{\mathbf{p}_1 - \mathbf{p}_0}{t_1 - t_0}$ e, pela atualização de Euler (exata aqui),

$$\mathbf{p}(t) = \mathbf{p}_0 + \mathbf{v}\,(t - t_0) = \mathbf{p}_0 + s\,(\mathbf{p}_1 - \mathbf{p}_0).$$

Componente a componente: $x = x_0 + s(x_1 - x_0)$, e igual para y e z.

Para ângulos não se interpola diretamente, porque $+179°$ e $-179°$ estão a $2°$ um do outro, não a $358°$. Calcula-se a diferença *no caminho curto*: $\Delta\psi = \operatorname{atan2}\big(\sin(\psi_1 - \psi_0), \cos(\psi_1 - \psi_0)\big)$, que está sempre em $[-\pi, \pi]$, e depois $\psi(t) = \psi_0 + s\,\Delta\psi$.

7 Câmara em 1.ª pessoa

A vista em 1.ª pessoa coloca a câmara virtual no drone. Para cada ponto do mundo calcula-se (a) onde está no referencial da câmara e (b) em que pixel aparece.

7.1 Referencial da câmara e ligação ao corpo

Os eixos da câmara são: x_c direita, y_c baixo, z_c frente (eixo ótico). Escritos nos eixos do corpo (frente, direita, baixo):

$$\hat{x}_c = \hat{y}_b = \begin{pmatrix}0\\1\\0\end{pmatrix}_b, \qquad \hat{y}_c = \hat{z}_b = \begin{pmatrix}0\\0\\1\end{pmatrix}_b, \qquad \hat{z}_c = \hat{x}_b = \begin{pmatrix}1\\0\\0\end{pmatrix}_b.$$

$\mathbf{R}_{bc}$ tem estes eixos como colunas:

$$\mathbf{R}_{bc} = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \qquad \mathbf{R}_{bc}^{\mathsf{T}} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}.$$

Conferência: $\mathbf{R}_{bc}^{\mathsf{T}}\,\mathbf{p}_b = (y_b,\, z_b,\, x_b)^{\mathsf{T}}$, isto é, $x_c = y_b$, $y_c = z_b$, $z_c = x_b$. Uma câmara a olhar em frente tem a profundidade z_c igual à distância à frente do drone.

Se a câmara estiver inclinada (por exemplo, a olhar para baixo), multiplica-se $\mathbf{R}_{bc}$ por uma rotação elementar adicional.

7.2 Do mundo para a câmara

Dedução 7.1 (Transformação completa mundo → câmara)**.** Da composição de referenciais, $\mathbf{p}_w = \mathbf{R}_{wc}\mathbf{p}_c + \mathbf{t}_{wc}$ com $\mathbf{R}_{wc} = \mathbf{R}_{wb}\mathbf{R}_{bc}$ e $\mathbf{t}_{wc} = \mathbf{t}_{wb} + \mathbf{R}_{wb}\mathbf{t}_{bc}$. Isolando $\mathbf{p}_c$ como na secção da vista local:

$$\boxed{\mathbf{p}_c = \mathbf{R}_{wc}^{\mathsf{T}}\,(\mathbf{p}_w - \mathbf{t}_{wc}) = \begin{pmatrix} x_c \\ y_c \\ z_c \end{pmatrix}}$$

7.3 Projeção em perspetiva (modelo pinhole)

Dedução 7.2 (De 3D para pixel)**.** Imagine-se a câmara como uma caixa com um orifício na origem do referencial da câmara e o plano de imagem a uma distância f à frente, perpendicular a z_c. Um ponto (x_c, y_c, z_c) projeta-se onde a reta que o une à origem corta o plano $z = f$.

Passo 1 — Triângulos semelhantes. Na vista lateral (plano y_c–z_c), o triângulo grande tem catetos z_c e y_c; o pequeno tem catetos f e y_{img}. São semelhantes (mesmos ângulos), logo

$$\frac{y_{\text{img}}}{f} = \frac{y_c}{z_c} \;\Rightarrow\; y_{\text{img}} = f\,\frac{y_c}{z_c}, \qquad \text{e analogamente} \qquad x_{\text{img}} = f\,\frac{x_c}{z_c}.$$

Passo 2 — Passar a pixéis. O pixel (u, v) conta-se a partir do canto superior esquerdo da imagem. Com f_x, f_y (distância focal em pixéis nas duas direções) e (c_x, c_y) o pixel central:

$$u = f_x\frac{x_c}{z_c} + c_x, \qquad v = f_y\frac{y_c}{z_c} + c_y.$$

Passo 3 — Forma matricial 3×3**.** Define-se a matriz intrínseca

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}.$$

Então $\mathbf{K}\mathbf{p}_c = \begin{pmatrix} f_x x_c + c_x z_c \\ f_y y_c + c_y z_c \\ z_c \end{pmatrix} = z_c \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}$. Ou seja:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{z_c}\,\mathbf{K}\,\mathbf{p}_c.$$

A divisão por z_c é o que faz objetos distantes parecerem pequenos.

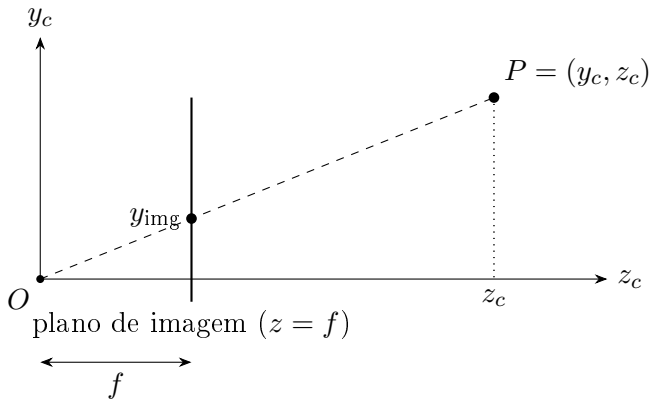


Figura 2: Triângulos semelhantes no modelo pinhole: $y_{\text{img}}/f = y_c/z_c$.

7.4 Distância focal a partir do campo de visão

Dedução 7.3 (f a partir do FOV horizontal)**.** Na borda direita da imagem, $u = W$ (largura em pixéis), com $c_x = W/2$, e o raio faz um ângulo FOV/2 com o eixo ótico: $x_c/z_c = \tan(\text{FOV}/2)$. De $u - c_x = f_x\, x_c/z_c$:

$$\frac{W}{2} = f_x \tan\frac{\text{FOV}}{2} \quad \Longrightarrow \quad f_x = \frac{W/2}{\tan(\text{FOV}/2)}.$$

Com pixéis quadrados, $f_y = f_x$.

7.5 Quando um ponto é visível

Um ponto só aparece se: $z_c > z_{\text{near}}$ (está à frente da câmara; $z_c \le 0$ é atrás e a divisão por z_c não faz sentido), $0 \le u < W$ e $0 \le v < H$, e opcionalmente $z_c < z_{\text{far}}$ (alcance de visibilidade na água).

7.6 Ligação a um motor gráfico 3D

Bibliotecas de visualização pedem a câmara como *posição*, *ponto para onde olha* e *vetor para cima*. Todos saem de $\mathbf{R}_{wc}$ e $\mathbf{t}_{wc}$ (colunas de $\mathbf{R}_{wc}$ = eixos da câmara no mundo):

$$\begin{aligned}
\text{posição} &= \mathbf{t}_{wc},\\
\text{direção de olhar} &= \text{coluna 3 de } \mathbf{R}_{wc}\ (\hat{z}_c \text{ no mundo}),\\
\text{ponto focal} &= \mathbf{t}_{wc} + d \cdot (\text{coluna 3}), \quad d > 0 \text{ qualquer},\\
\text{vetor para cima} &= -(\text{coluna 2 de } \mathbf{R}_{wc}) \quad (\text{pois } y_c \text{ aponta para baixo}).
\end{aligned}$$

Se o motor trabalhar com z para cima, aplica-se uma conversão fixa NED $\to$ mundo do motor (por exemplo, $x' = x$, $y' = y$, $z' = -z$ muda z de baixo para cima; trocar um eixo inverte a mão do sistema, por isso confirme com um teste visual).

8 Exemplo numérico completo

Drone em $\mathbf{t}_{wb} = (10,\, 5,\, 20)^{\mathsf{T}}$ m (20 m de profundidade), com $\psi = 90°$ (a olhar para Este), $\theta = \phi = 0$. Câmara na origem do corpo, a olhar em frente, imagem 640×480, FOV horizontal $90°$.

Intrínsecos. $f_x = \dfrac{320}{\tan 45^\circ} = 320$, $f_y = 320$, $c_x = 320$, $c_y = 240$.

Orientação. Com $\psi = 90^\circ$: $c_\psi = 0$, $s_\psi = 1$, logo

$$\mathbf{R}_{wb} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \mathbf{R}_{wb}^{\mathsf{T}} = \begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Como $\mathbf{t}_{bc} = \mathbf{0}$, $\mathbf{t}_{wc} = \mathbf{t}_{wb}$.

Ponto $P_1 = (10,\, 15,\, 20)^{\mathsf{T}}$ (10 m a Este, à mesma profundidade).

$$\begin{aligned} \mathbf{p}_w - \mathbf{t}_{wc} &= (0,\, 10,\, 0)^{\mathsf{T}}, \\ \mathbf{p}_b &= \mathbf{R}_{wb}^{\mathsf{T}}(0, 10, 0)^{\mathsf{T}} = (10,\, 0,\, 0)^{\mathsf{T}} \quad \text{(10 m em frente)}, \\ \mathbf{p}_c &= \mathbf{R}_{bc}^{\mathsf{T}}\mathbf{p}_b = (y_b, z_b, x_b)^{\mathsf{T}} = (0,\, 0,\, 10)^{\mathsf{T}}, \\ u &= 320 \cdot \tfrac{0}{10} + 320 = 320, \qquad v = 320 \cdot \tfrac{0}{10} + 240 = 240. \end{aligned}$$

O ponto aparece no centro da imagem, como esperado.

Ponto $P_2 = (10,\, 15,\, 25)^{\mathsf{T}}$ (igual, mas 5 m mais fundo).

$$\begin{aligned} \mathbf{p}_w - \mathbf{t}_{wc} &= (0,\, 10,\, 5)^{\mathsf{T}}, \\ \mathbf{p}_b &= \mathbf{R}_{wb}^{\mathsf{T}}(0, 10, 5)^{\mathsf{T}} = (10,\, 0,\, 5)^{\mathsf{T}}, \\ \mathbf{p}_c &= (0,\, 5,\, 10)^{\mathsf{T}}, \\ u &= 320, \qquad v = 320 \cdot \tfrac{5}{10} + 240 = 400. \end{aligned}$$

Aparece centrado na horizontal e na parte inferior da imagem ($v = 400$ está abaixo do centro), o que é correto: está mais fundo do que o drone.

9 Arquitetura do software

9.1 Classes C++ e a matemática que implementam

Classe	Responsabilidade	Secção
`Vec3`	vetor (x, y, z), soma, produto escalar e vetorial, norma	1, 3
`Mat3`	matriz 3×3, produto, transposta, $\times$ vetor	4
`Rotation`	construção por ϕ, θ, ψ; extração de ângulos; Rodrigues; re-ortonormalização	4, 6
`Pose`	par $(\mathbf{R},\, \mathbf{t})$: transformar ponto, inverter, compor	5
`Geodesy`	WGS84: LLA$\to$ECEF$\to$NED$\to$AER	3
`Camera`	$\mathbf{K}$, $\mathbf{R}_{bc}$, projeção, visibilidade, vetores look-at	7
`Trajectory`	amostras no tempo, interpolação, orientação pela velocidade	6

Tabela 2: Mapa entre a matemática e o código.

9.2 Esboço da interface

```
class Vec3 {
public:
    double x, y, z;
    Vec3 operator+(const Vec3& o) const;
    double dot(const Vec3& o) const;
    Vec3 cross(const Vec3& o) const;
    double norm() const;
};

class Mat3 {
public:
    double m[3][3]; // linha, coluna
    Mat3 operator*(const Mat3& o) const;
    Vec3 operator*(const Vec3& v) const;
    Mat3 transpose() const; // = inverse, if rotation
};

class Pose {
public:
    Mat3 R; Vec3 t; // R_wb, t_wb
    Vec3 toWorld(const Vec3& p_b) const { return R * p_b + t; }
    Vec3 toBody (const Vec3& p_w) const { return R.transpose() * (p_w - t); }
};

class Camera {
public:
    double fx, fy, cx, cy;
    Pose bodyToCamera; // R_bc, t_bc
    bool project(const Pose& drone, const Vec3& p_w,
                 double& u, double& v) const; // false if behind camera
};
```

Os métodos `toWorld` e `toBody` implementam diretamente as equações da Secção 5; `project` aplica a cadeia mundo→câmara e depois a divisão por z_c da Secção 7.

9.3 Ligação Python ↔ C++

As classes são expostas ao Python com `pybind11`. Do lado do Python, o código de aplicação só chama a biblioteca (por exemplo, `pose.to_body(p)`) e fica sem matemática própria.

9.4 Testes que validam a matemática

Cada propriedade deduzida neste documento vira um teste automático:

- $\mathbf{R}^\top\mathbf{R} = \mathbf{I}$ e $\det \mathbf{R} = 1$ para rotações aleatórias;
- ida e volta: `toBody(toWorld(p))`$= \mathbf{p}$;
- ângulos → matriz → ângulos devolve os mesmos ângulos (fora do gimbal lock);
- o Exemplo da Secção 8 dá exatamente $(u, v) = (320, 240)$ e $(320, 400)$;
- Rodrigues com $\omega = (0, 0, 1)^\top$ e $\Delta t = \pi/2$ dá $\mathbf{R}_z(90°)$.

10 Resumo das fórmulas

Operação	Fórmula
Rotação do corpo	$\mathbf{R}_{wb} = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$
Corpo → mundo	$\mathbf{p}_w = \mathbf{R}_{wb}\mathbf{p}_b + \mathbf{t}_{wb}$
Mundo → corpo (Local View)	$\mathbf{p}_b = \mathbf{R}_{wb}^{\mathsf{T}}(\mathbf{p}_w - \mathbf{t}_{wb})$
Mundo → câmara	$\mathbf{p}_c = (\mathbf{R}_{wb}\mathbf{R}_{bc})^{\mathsf{T}}(\mathbf{p}_w - \mathbf{t}_{wc})$
Projeção	$u = f_x x_c/z_c + c_x, \quad v = f_y y_c/z_c + c_y$
Focal	$f_x = \dfrac{W/2}{\tan(\mathrm{FOV}/2)}$
Posição (Euler)	$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t$
Orientação (EDO)	$\dot{\mathbf{R}} = \mathbf{R}[\omega_b]_\times$
Rodrigues	$\mathbf{I} + \sin\alpha\,\mathbf{K} + (1-\cos\alpha)\mathbf{K}^2$
Yaw/pitch pela velocidade	$\psi = \mathrm{atan2}(v_y, v_x)$, $\theta = \mathrm{atan2}(-v_z, \sqrt{v_x^2 + v_y^2})$
Interpolação	$\mathbf{p} = \mathbf{p}_0 + s(\mathbf{p}_1 - \mathbf{p}_0)$